

Text as Data

Session 2: Basics in R, Part 1

Mirko Wegemann

University of Münster
Institute for Political Science

22 April 2026

What is R?



R is an open-source programming language that enables us in the social sciences to perform and visualize statistical analyses.

It is available for Windows as well as Mac (+ UNIX).

What is RStudio?



R is a programming language. We can use it in the OS console.

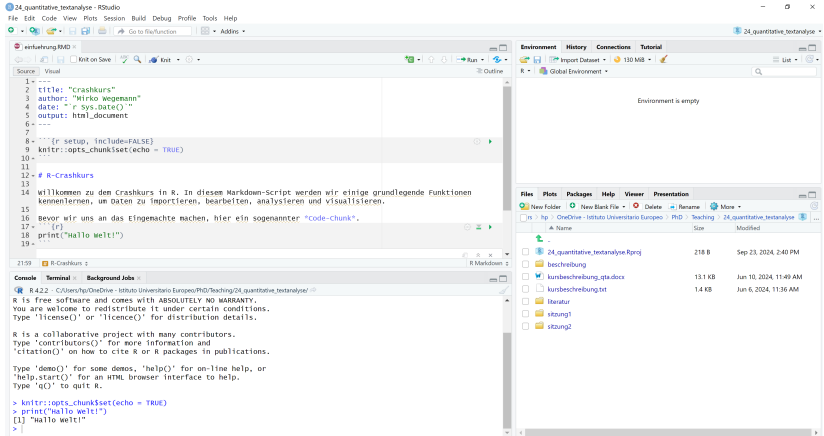
RStudio also provides a graphical interface. It shows us, for example, graphics and environment variables.

Installation of RStudio

Our software environment can be set up in the following steps:

1. Download and install R
2. Download and install RStudio
3. For now optional, but later useful for some packages: RTools

Structure of RStudio I



The screenshot displays the RStudio environment with the following components:

- Source Editor:** Contains an R script with a YAML header for a document, followed by R code to set up the chunk and print a message.


```
1 ---
2 title: "Crashkurs"
3 author: "Mirko Wegemann"
4 date: "r_sys.Date()"
5 output: html_document
6 ---
7
8 ---[r setup, include=FALSE]
9 knitr::opts_chunk$set(echo = TRUE)
10 ---
11
12 # R-Crashkurs
13
14 Willkommen zu den Crashkurs in R. In diesem Markdown-Script werden wir einige grundlegende Funktionen
15 kennenlernen, um Daten zu importieren, bearbeiten, analysieren und visualisieren.
16
17 Bevor wir uns an das Eingensichte machen, hier ein sogenannter "Code-Chunk".
18
19 print("Hallo Welt!")
```
- Console:** Shows the output of the R script, including the R version (4.2.2) and the printed message "Hallo Welt!".


```
R 4.2.2 - C:\Users\hp\OneDrive - Istituto Universitario Europeo\PhD\Teaching\24_quantitative_textanalyse\
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> knitr::opts_chunk$set(echo = TRUE)
> print("Hallo Welt!")
[1] "Hallo Welt!"
```
- Environment:** Shows the current environment is empty.
- Files:** A file explorer showing the project structure, including folders like "beschreibung" and "literatur", and files like "24_quantitative_textanalyse.Rproj", "kursbeschreibung_qta.docx", and "kursbeschreibung.tex".

Structure of RStudio II

R consists of four **Workspace Panes**.

1. **Source** → this is where we write our code
2. **Console** → this is where we see the output of our code
3. **Environment** → this shows all objects that we have created with our code
4. **Files/Plots/Packages/Help** → here we see, among other things, the files that are in the same folder structure or the graphics we generate

Why do we need R?

- Importing files (datasets in various formats, e.g., .xlsx, .csv, .dta, .spss, .R, .RDS, .txt) → this week
- Data preparation (e.g., renaming and creating variables, aggregating data, filtering datasets) → this week
- Visualization → this week
- Regression analyses → this week
- Communicating with websites (e.g., web-scraping) → functions or loops (*for-loops*) → later
- Text analysis (e.g., via dictionaries, topic models, scaling models, neural networks) → later

Tutorial

Open the tutorial. The first part of the tutorial is to understand object types in R. The second part covers data wrangling. For the second part, you can switch between the slides and the tutorial. The slides explain basic functions of `dplyr()`.

For any further assistance, refer to Wickham et al. (2023) or the R help file (`?command`).

Functions in R

With the installation of R we already have access to a large number of pre-installed functions (the so-called *base R*).

- Some analyses (such as our text analyses) however require additional user-written functions
- freely available through *CRAN*, which usually also provides extensive documentation for the packages
- in R we install these packages with the command `install.packages(PACKAGENAME)` and then load them into the workspace with `library(PACKAGENAME)`

An example:

```
1  
2 > install.packages("tidyverse")  
3 > library("tidyverse")
```

Importing Data Sets

R can handle a wide variety of file formats. For this week, we only focus on `read.csv()`.

```
1  
2 data <- read.csv("./session2/ess11.csv")  
3 View(data)
```

Different Methods I

In R, we have different methods for modifying data

- *base R*: the pre-programmed original method for transforming vectors
- *dplyr* (from *tidyverse*): a customized, simplified method that focuses on code readability
- *data.table*: similar to *base R* in syntax, but specialized for efficiency

Different Methods II

All methods have their advantages and disadvantages. We will primarily work with *dplyr* in this seminar due to its relative simplicity. However, depending on the size of the data structures, we will also show applications of *data.table*.

The Pipe Structure

dplyr is known for its pipe structure. The idea is that we can perform different options sequentially on an object (e.g., a data frame).

- Pipe structures always start with the name of the object we want to modify
- At the end of each command, a pipe `%>%` **or** `|>` is used

Generating a New Variable

You can create a new variable using the *mutate* function

```
1 df <- df %>%  
2 mutate(new_var = "value")
```

Changing/Generating New Variables with Conditions

We often create new variables that are transformations of existent variables. For example, we could categorize an income variable that was previously asked openly.

The *case_when()* function is very helpful for this purpose.

```
1 df <- df %>%  
2 mutate(inc_cat = case_when(inc<1000 ~ "<1000  
   Euro",  
3 inc>=1000 & inc<2000 ~ "1000-2000 Euro",  
4 inc>=2000 & inc<3000 ~ "2000-3000 Euro",  
5 inc>=3000 ~ ">3000 Euro"))
```

Filtering the Dataset

A raw dataset sometimes contains cases that we don't need for our analysis. We can remove them using the *filter()* function. In the example below, we restrict our dataset to cases from eastern German states.

```
1 df <- df %>%  
2 filter(eastwest=="NEW FEDERAL STATES")
```


Aggregating Data

dplyr package lets us group data using *group_by()* we can then aggregate it using *summarise()*.

```
1 df_aggr <- df %>%  
2   group_by(eastwest) %>%  
3   summarise(average_income = mean(inc, na.rm=T))
```

Common operations include mean (*mean()*), median (*median()*), minimum (*min()*), maximum (*max()*) or simply counting a category (*n()*).

Selecting a Specific Variable

The *select()*-function shows us not the entire dataset, but only one or more target variables. In this case, we previously sorted the variable using *arrange()*.

```
1 df4 %>%  
2   arrange(desc(inc)) %>%  
3   select(inc)
```

Data Manipulation

In the file *introduction.RMD*, you will find some blank spaces starting from line 145. Let's try to fill them in together.

Visualization in R

In R, there are several ways to visualize distributions and relationships. We will primarily use the *ggplot2* package from the *tidyverse*.

A Histogram

The structure of `ggplot` is simple. We can use the pipe to first define the dataset from which the target variable (in this case *inc*) comes. Then, in the main command, we define the *aesthetics*, i.e., the variables we want to plot. Using a plus sign, we can choose from one or more visualization forms, in this case, a histogram.

```
1 df4 %>%  
2   ggplot(aes(inc)) +  
3   geom_histogram()
```

Bivariate Relationships

Visualizations are particularly helpful when we want to display bivariate relationships (associations between two variables). Here, we plot a scatterplot **and** a regression line (in this case, based on local effect sizes).

```
1 df4 %>%  
2   ggplot(aes(hhinc, inc)) +  
3   geom_point() + geom_smooth()
```

Regression Analysis

In R, multivariate regression analysis can be performed easily. There are many packages, depending on the model underlying our data.

- $lm(y \sim x1 + x2, data)$ estimates a multivariate linear regression
- $glm(y \sim x1 + x2, data, family="binomial")$ estimates a logistic regression

A Linear Regression Model

In the following example, a linear regression model is estimated for the human capital theory. Income is a function of education and work experience.

```
1  m1 <- lm(inc ~ educ + experience, data)
2  summary(m1)
```


Literature I

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science*. O'Reilly Media, Inc.